

ANN4: Perceptron Decision Boundary – Detailed Code & Viva Explanation

Cell 1: Import Libraries

Code:

```
import numpy as np
import matplotlib.pyplot as plt
```

Explanation (what to say in viva):

Here I am importing numpy and matplotlib libraries.

Numpy is used for mathematical calculations.

Matplotlib is used to visualize data using graphs.

Cell 2: Define Dataset

Code:

```
x = np.array([[2,3],[4,5],[1,1],[6,7]])
y = np.array([0,0,1,1])
```

Explanation (what to say in viva):

X contains 2D input points.

y contains class labels.

These points belong to two different classes.

This is supervised learning.

Cell 3: Initialize Parameters

Code:

```
w = np.zeros(2)
b = 0
alpha = 0.1
```

Explanation (what to say in viva):

w represents weights for two inputs.

b is bias.

alpha is learning rate.

These are initial values before training.

Cell 4: Training Loop

Code:

```
for _ in range(10):
    for i in range(len(X)):
        z = np.dot(X[i], w) + b
        y_hat = 1 if z >= 0 else 0
        w = w + alpha*(y[i]-y_hat)*X[i]
        b = b + alpha*(y[i]-y_hat)
```

Explanation (what to say in viva):

This loop trains the perceptron model.
z is weighted sum.
y_hat is predicted output.
Error is calculated and weights updated.
This helps model learn separation.

Cell 5: Plot Points

Code:

```
plt.scatter(X[:,0], X[:,1], c=y)
```

Explanation (what to say in viva):

This plots input points on graph.
Different colors represent different classes.
This helps visualize data distribution.

Cell 6: Decision Boundary

Code:

```
x1 = np.linspace(0,8)
x2 = -(w[0]*x1 + b)/w[1]
plt.plot(x1,x2)
```

Explanation (what to say in viva):

x1 creates range of values.
x2 calculates corresponding boundary.
Equation derived from $w.x + b = 0$.
This line separates two classes.

Cell 7: Show Graph

Code:

```
plt.show()
```

Explanation (what to say in viva):

This displays the graph.
It shows data points and decision boundary.
This helps understand classification visually.